

✓ Laboratorio 4

- Diego Valenzuela 22309
- Daniel Dubon 22233
- Nelson García Bravatti 22434
- Joaquin Puente 22296

✓ Task 1 Diseño

1. Espacio de Estados y Espacio de Acciones

Espacio de Estados: El almacén es una cuadrícula de 8 x 8.

Elección: gymnasium spaces Discrete 64

Justificación: Un espacio Discreto 64 mapea cada celda bidimensional a un único número entero entre 0 y 63 y esto es ideal y necesario porque tanto SARSA como Q Learning, en sus formas clásicas tabulares, requieren un estado discreto para usarse como índice en la matriz o Q Table de dimensiones 64 x 4.

Por qué no usar Box: La clase Box se utiliza para representar espacios continuos como arreglos de números decimales. Si se usara Box el espacio de estados sería infinito y no se podría construir una Q Table clásica y nos veríamos obligados a usar aproximadores de funciones lo cual está fuera del alcance de una comparación estándar de SARSA contra Q Learning.

Espacio de Acciones: Movimientos direccionales del robot.

Elección: gymnasium spaces Discrete 4

Justificación: Representa 4 acciones deterministas y mutuamente excluyentes: Arriba 0, Abajo 1, Izquierda 2, Derecha 3.

2. Función de Recompensa

- Recompensa por entrega exitosa: +100
- Penalización por zona de congestión: -20 y se aplica cada vez que se pisa o permanece en una celda de congestión.
- Penalización por paso: -1 y se aplica en cualquier otra celda transitable.

Justificación de las magnitudes:

El objetivo es que el robot llegue a la meta de la forma más rápida por la penalización por paso y segura por la penalización por congestión y aumentar 100 garantiza que llegar a la meta sea el evento dominante y el objetivo global del episodio. El menos 1 fuerza al robot a buscar la ruta más corta y el menos 20 es lo suficientemente grande como para disuadir al robot de cruzar la congestión también cabe resaltar que cruzar una celda de congestión cuesta lo mismo que dar 20 pasos normales.

Comportamientos indeseables por mala ponderación:

Si la penalización por congestión es menor o igual a la penalización por paso el robot desarrollará un comportamiento temerario y si rodear la congestión toma 3 pasos extra pero cruzarla cuesta menos puntos el agente preferirá atravesar la congestión. Si la penalización por paso es cero o positiva el robot perderá el sentido de urgencia y caminará en círculos infinitamente sin incentivo para llegar a la meta. Si la penalización por congestión es excesivamente alta, por ejemplo menos 1000: Podría provocar que durante la exploración temprana el agente asocie todo el entorno con un peligro extremo y aprenda una política subóptima de quedarse quieto o chocar contra la pared constantemente para terminar el episodio rápidamente y evitar castigos.

3. Condición de Terminación del Episodio

El episodio terminará por cualquiera de estas dos condiciones:

- Terminated o Estado Terminal Natural: El robot alcanza el estado del Punto de Entrega.
- Truncated o Límite Máximo de Pasos: El episodio alcanza un límite, por ejemplo un máximo de 100 pasos.

Es apropiado tener un límite máximo de pasos:

Si, es necesario porque durante las primeras fases de entrenamiento la política del robot es casi completamente aleatoria y sin un límite de pasos el robot podría entrar en bucles infinitos en zonas sin penalizaciones, o simplemente vagar sin encontrar la meta jamás y el límite asegura que los episodios finalicen, garantizando que el agente experimente repetidas veces el reinicio del entorno y converja matemáticamente.

4. Diseño del Mapa 8 por 8

Para que la diferencia entre una política agresiva y una política conservadora sea empíricamente visible diseñamos el mapa inspirándonos en el problema clásico de Cliff Walking.

Leyenda de las celdas: S: Punto de recogida G: Punto de entrega P: Pasillo transitable O: Obstáculo fijo C: Zona de congestión

Esquema de las primeras 4 filas:

--

S P P P P P G

O C C C C C C P

P P P P P P P

P O O P O O P

--

Análisis de la configuración espacial:

En este mapa la primera fila representa una ruta directa muy rápida entre el punto de recogida y la meta, sin embargo justo debajo hay una hilera de Zonas de Congestión y si un robot está en la fila de arriba y toma una decisión aleatoria hacia abajo caerá en la congestión, existe una segunda ruta segura por debajo de los obstáculos pero toma muchos más pasos.

Comportamiento diferencial esperado:

Con Q Learning Agresivo y Off Policy esperamos que aprenda la función de valor óptima asumiendo que actuara siempre de forma ambiciosa, en este caso aprenderá que la ruta óptima es ir en línea recta por la fila de arriba, sin embargo durante el entrenamiento con mucha exploración, su constante curiosidad hará que resbale frecuentemente hacia abajo sufriendo muchas penalizaciones y su política es óptima en la teoría pero altamente riesgosa en la práctica.

Con SARSA conservador y On Policy: Aquí se actualizan sus valores basándose en la acción real que va a tomar incluyendo los movimientos aleatorios de exploración donde SARSA aprende que caminar por la fila de arriba es peligroso porque hay probabilidad de terminar en la congestión, por lo tanto SARSA convergerá hacia una ruta más larga pero segura, bajando para alejarse de la zona de congestión, minimizando las penalizaciones durante el entrenamiento a costa de dar más pasos.

Preguntas previas a la implementación

1. Predicción formal: recompensa acumulada en entrenamiento vs. evaluación greedy

Las reglas de actualización son:

- SARSA (on-policy): $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma Q(s', a') - Q(s, a)]$, donde a' es la acción realmente elegida por la política ε -greedy en s' .
- Q-Learning (off-policy): $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$, sin importar qué acción se ejecute realmente.

En nuestro mapa, la fila 0 (ruta corta S→G) es adyacente a una fila completa de congestión (fila 1). Para cualquier estado de la fila 0, la acción "abajo" cae en congestión (-20).

- Durante **entrenamiento**, SARSA incorpora en su target el valor esperado bajo la política que realmente se ejecuta, es decir, con probabilidad ε el agente puede tomar una acción aleatoria y caer en congestión. Esto hace que $Q(s, \text{arriba}/\text{derecha})$ de los estados de la fila 0 quede penalizado por ese riesgo, empujando a SARSA a preferir la ruta segura de la fila 2 (más pasos, pero sin exposición a -20). Resultado: **SARSA acumula mayor recompensa durante el entrenamiento** porque su política de comportamiento evita activamente la congestión mientras explora.
- Q-Learning usa $\max_{a'} Q(s', a')$ en el target, ignorando que la acción real puede ser exploratoria. Por lo tanto, aprende que la fila 0 es óptima (siempre y cuando actúe greedy), pero mientras entrena con ε -greedy sigue tomando pasos aleatorios que lo tiran a la congestión con frecuencia. Resultado: **Q-Learning tiene menor recompensa acumulada durante el entrenamiento** (más caídas en zona de congestión, análogo al problema clásico de "Cliff Walking").
- Durante **evaluación con política greedy pura** ($\varepsilon = 0$), ya no hay exploración: el entorno es determinista, así que no hay riesgo de "resbalar". Q-Learning ejecuta la ruta óptima real (fila 0, distancia mínima = 7 pasos), obteniendo la recompensa más alta posible. SARSA, en cambio, quedó convergido a la ruta conservadora de la fila 2 (más larga), por lo que su recompensa en evaluación greedy es menor que la de Q-Learning aunque siga siendo positiva y estable.

En resumen: **SARSA gana en entrenamiento, Q-Learning gana en evaluación greedy**. Esto es exactamente el resultado esperado por la propiedad on-policy vs. off-policy combinada con la geometría tipo "cliff" del mapa.

2. Efecto de ε y condición de convergencia a políticas idénticas

El target de SARSA puede escribirse como una esperanza sobre la política de comportamiento:

$$\mathbb{E}_{a' \sim \pi_\varepsilon}[Q(s', a')] = (1 - \varepsilon) \max_{a'} Q(s', a') + \frac{\varepsilon}{|A|} \sum_{a'} Q(s', a')$$

mientras que el target de Q-Learning es directamente $\max_{a'} Q(s', a')$.

La diferencia entre ambos targets es:

$$\Delta = \max_{a'} Q(s', a') - \mathbb{E}_{a' \sim \pi_\varepsilon}[Q(s', a')] = \varepsilon \left(\max_{a'} Q(s', a') - \frac{1}{|A|} \sum_{a'} Q(s', a') \right)$$

Esta expresión es **lineal en** ε y su segundo factor es siempre ≥ 0 (el máximo nunca es menor que el promedio). Por lo tanto:

- Cuanto mayor es ε , mayor es la brecha entre SARSA y Q-Learning, y más conservadora se vuelve la política de SARSA respecto a la fila de congestión.
- Cuando $\varepsilon \rightarrow 0$, $\Delta \rightarrow 0$: las dos reglas de actualización se vuelven **matemáticamente idénticas** ($\pi_\varepsilon \rightarrow \text{greedy}$), por lo que ambos algoritmos convergen a la misma política óptima determinista π^* (la ruta por la fila 0).

Si existe entonces un valor de ε para el cual ambos convergen a políticas idénticas: $\varepsilon = 0$ (equivalente a que ε decaiga a 0 durante el entrenamiento, cumpliendo condiciones GLIE). La salvedad es que con $\varepsilon = 0$ fijo desde el inicio no hay exploración, por lo que la garantía de convergencia al óptimo requiere que ε decaiga gradualmente (arrancando > 0 para explorar y tendiendo a 0), no que sea 0 desde el primer episodio.

3. Cota superior de $V^*(s_0)$

Asumiendo que el agente sigue siempre la ruta más corta sin pisar congestión, la ruta óptima es la fila 0 completa:

$S = (0, 0) \rightarrow G = (0, 7)$, con distancia Manhattan $d = 7$ pasos (sin obstáculos ni congestión en esa fila).

De esos 7 pasos, los primeros $d - 1 = 6$ caen en celdas de pasillo (-1 cada uno) y el último paso entra a la meta (+100). Con factor de descuento γ , la cota superior es:

$$V^*(s_0) \leq \sum_{t=0}^{d-2} \gamma^t (-1) + \gamma^{d-1} (100) = -\left(\frac{1 - \gamma^6}{1 - \gamma}\right) + 100 \gamma^6$$

Casos concretos:

- Sin descuento ($\gamma = 1$): $V^*(s_0) \leq 6(-1) + 100 = 94$.
- Con $\gamma = 0.99$: $V^*(s_0) \leq -5.853 + 100(0.941) \approx 88.3$.
- Con $\gamma = 0.9$: $V^*(s_0) \leq -4.686 + 100(0.531) \approx 48.4$.

Esta cota (94 para el caso no descontado, que es el que usamos en la implementación tabular) sirve como referencia: cualquier retorno de evaluación greedy que obtengamos de SARSA o Q-Learning debe estar por debajo o igual a 94, y qué tan cerca esté cada algoritmo de ese valor mide su cercanía al óptimo teórico.

Task 3

Corrección al mapa diseñado en Task 1

Al implementar el mapa tal como quedó escrito en Task 1 detectamos dos problemas que lo hacían inservible para comparar SARSA contra Q-Learning, y los verificamos empíricamente entrenando ambos algoritmos sobre las variantes:

1. **No existía la "segunda ruta segura"**. Con la fila 1 definida como `0 C C C C C C P`, la celda `(1, 0)` era un obstáculo, así que desde `S=(0, 0)` el único movimiento posible era avanzar por la fila 0. Toda bajada intermedia caía en congestión y la única celda transitable de la fila 1 (`(1, 7)`) está debajo de la meta, a la que solo se llega terminando el episodio. En la práctica el mapa tenía **una sola ruta**, y ambos algoritmos convergían necesariamente a la misma política.
2. **La ruta alternativa estaba igual de expuesta**. Aun corrigiendo `(1, 0)`, la ruta "segura" por la fila 2 está pegada por debajo a la misma hilera de congestión, con exactamente la misma probabilidad de resbalar hacia ella que la fila 0. Al ser 4 pasos más larga sin reducir el riesgo, SARSA nunca tenía motivo para preferirla. Lo comprobamos con tres variantes de mapa, tres valores de penalización y tres valores de ε : en todos los casos ambos algoritmos aprendían la misma política greedy.

La causa de fondo es que en el mapa original la congestión quedaba **al costado** de la ruta corta, mientras que en el problema de Cliff Walking en el que nos inspiramos el peligro está **entre** el inicio y la meta. Reacomodamos el mapa para respetar esa geometría, manteniendo intactos el espacio de estados (`Discrete(64)`), el de acciones (`Discrete(4)`) y la función de recompensa (+100 meta, -20 congestión, -1 paso):

```
S ~ ~ ~ ~ ~ G      fila 0: recogida, banda de congestion, entrega
. . . . .          fila 1: ruta corta y arriesgada (9 pasos, pegada a la congestion)
```

```

. . . . .      fila 2: ruta larga y segura (11 pasos, sin exposicion)
. # # . # # # .      fila 3: obstaculos fijos
. . . . .
. # # # # # .
. . . . .
. . . . .

```

Con este mapa el comportamiento diferencial que predijimos sí aparece: la ruta óptima real (fila 1) bordea la congestión, y solo cuesta 2 pasos más rodearla por la fila 2.

Consecuencia sobre la cota de Task 2: la ruta óptima ya no mide 7 pasos sino 9, así que la cota no descontada pasa de 94 a

$8(-1) + 100 = 92$. Con $\gamma = 0.99$ la cota es $-\left(\frac{1-0.99^8}{1-0.99}\right) + 100(0.99^8) \approx 84.6$. El resto del análisis de Task 2 (on-policy vs off-policy, efecto de ε , convergencia idéntica en $\varepsilon = 0$) no cambia.

Hiperparámetros usados: $\alpha = 0.1$, $\gamma = 0.99$, $\varepsilon = 0.2$ fijo, 20000 episodios de entrenamiento, límite de 100 pasos por episodio y 100 episodios de evaluación greedy pura ($\varepsilon = 0$).

Dos de estos valores los elegimos a partir de un control de robustez sobre 10 semillas distintas, no por conveniencia de una corrida puntual:

- $\varepsilon = 0.2$. Con $\varepsilon = 0.1$ el riesgo esperado de resbalar (≈ 0.15 caídas $\times \approx 21$ puntos) no alcanza a compensar los 2 pasos extra del rodeo, y ambos algoritmos vuelven a converger a la misma política. Esto es consistente con el análisis de Task 2: la brecha entre los targets de SARSA y Q-Learning es lineal en ε , así que hace falta suficiente exploración para que la aversión al riesgo de SARSA se manifieste.
- **20000 episodios.** Con 8000 episodios SARSA elegía la ruta segura solo en 8 de 10 semillas (en las otras 2 todavía no había terminado de converger y coincidía con Q-Learning). Con 20000 episodios el resultado es 10 de 10 semillas, y SARSA supera a Q-Learning en recompensa de entrenamiento en las 10. Q-Learning converge antes: aprende la ruta corta en 10 de 10 semillas ya con 8000 episodios. Esa diferencia de velocidad de convergencia es en sí misma un resultado del experimento.

1. El entorno: WarehouseEnv

```

1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from matplotlib.colors import ListedColormap
5
6 import gymnasium as gym
7 from gymnasium import spaces
8
9 # Leyenda: S=recogida, G=entrega, P=pasillo, O=obstaculo fijo, C=zona de congestion
10 MAPA = [
11     "SCCCCCCG",
12     "PPPPPPPP",
13     "PPPPPPPP",
14     "POOP000P",
15     "PPPPPPPP",
16     "PO00000P",
17     "PPPPPPPP",
18     "PPPPPPPP",
19 ]
20
21 R_META = 100.0
22 R_CONGESTION = -20.0
23 R_PASO = -1.0
24
25
26 class WarehouseEnv(gym.Env):
27     """Almacén 8x8 para un robot de reparto.
28
29     Observacion : Discrete(64), índice del estado = fila * 8 + columna.
30     Accion      : Discrete(4) -> 0 arriba, 1 abajo, 2 izquierda, 3 derecha.
31     Recompensa  : +100 al entrar a la meta, -20 al pisar o permanecer en congestion,
32                  -1 en cualquier otro caso.
33     Terminacion : terminated cuando el robot entra a la meta.
34                  truncated cuando se agota el límite de pasos del episodio.
35
36     """
37     metadata = {"render_modes": ["human", "ansi", "matplotlib"], "render_fps": 4}
38
39     ACCIONES = {0: (-1, 0), 1: (1, 0), 2: (0, -1), 3: (0, 1)}

```

```

40 NOMBRES_ACCIONES = {0: "arriba", 1: "abajo", 2: "izquierda", 3: "derecha"}
41 FLECHAS = {0: "^", 1: "v", 2: "<", 3: ">"}
42
43 def __init__(self, mapa=MAPA, max_pasos=100, render_mode=None):
44     super().__init__()
45     self.mapa = [list(fila) for fila in mapa]
46     self.n_filas = len(self.mapa)
47     self.n_cols = len(self.mapa[0])
48     self.max_pasos = max_pasos
49     self.render_mode = render_mode
50
51     self.observation_space = spaces.Discrete(self.n_filas * self.n_cols)
52     self.action_space = spaces.Discrete(4)
53
54     self.inicio = self._buscar("S")
55     self.meta = self._buscar("G")
56
57     self.pos = self.inicio
58     self.pasos = 0
59
60 # ----- utilidades
61 def _buscar(self, simbolo):
62     for f in range(self.n_filas):
63         for c in range(self.n_cols):
64             if self.mapa[f][c] == simbolo:
65                 return (f, c)
66             raise ValueError(f"El mapa no contiene el simbolo {simbolo}")
67
68 def a_estado(self, pos):
69     """Convierte una posicion (fila, columna) al indice discreto del estado."""
70     return pos[0] * self.n_cols + pos[1]
71
72 def a_posicion(self, estado):
73     """Convierte un indice discreto de estado a la posicion (fila, columna)."""
74     return divmod(estado, self.n_cols)
75
76 def _transitable(self, pos):
77     f, c = pos
78     if not (0 <= f < self.n_filas and 0 <= c < self.n_cols):
79         return False
80     return self.mapa[f][c] != "0"
81
82 # ----- API Gymnasium
83 def reset(self, seed=None, options=None):
84     super().reset(seed=seed)
85     self.pos = self.inicio
86     self.pasos = 0
87     info = {"pos": self.pos, "congestion": False, "pasos": 0}
88     if self.render_mode == "human":
89         self.render()
90     return self.a_estado(self.pos), info
91
92 def step(self, accion):
93     assert self.action_space.contains(accion), f"Accion invalida: {accion}"
94     df, dc = self.ACCIONES[int(accion)]
95     candidata = (self.pos[0] + df, self.pos[1] + dc)
96
97     # Chocar contra una pared o un obstaculo deja al robot donde estaba.
98     if self._transitable(candidata):
99         self.pos = candidata
100
101     self.pasos += 1
102     celda = self.mapa[self.pos[0]][self.pos[1]]
103
104     terminated = celda == "G"
105     if terminated:
106         recompensa = R_META
107     elif celda == "C":
108         recompensa = R_CONGESTION # se cobra al pisar y al permanecer
109     else:
110         recompensa = R_PASO
111
112     truncated = (not terminated) and self.pasos >= self.max_pasos
113     info = {"pos": self.pos, "congestion": celda == "C", "pasos": self.pasos}
114
115     if self.render_mode == "human":
116         self.render()

```

```

117         return self.a_estado(self.pos), recompensa, terminated, truncated, info
118
119     # ----- render
120     def render(self):
121         modo = self.render_mode or "ansi"
122         if modo == "matplotlib":
123             return self._render_matplotlib()
124         texto = self._render_texto()
125         if modo == "human":
126             print(texto)
127             return None
128         return texto
129
130     def _render_texto(self):
131         simbolos = {"S": " S ", "G": " G ", "P": " . ", "O": " # ", "C": " ~ "}
132         lineas = [f"paso {self.pasos}/{self.max_pasos}   pos={self.pos}"]
133         for f in range(self.n_filas):
134             fila = ""
135             for c in range(self.n_cols):
136                 fila += "[A]" if (f, c) == self.pos else simbolos[self.mapa[f][c]]
137             lineas.append(fila)
138         lineas.append("A=agente S=recogida G=entrega #=obstaculo ~=congestion .=pasillo")
139         return "\n".join(lineas)
140
141     def _grid_codificado(self):
142         codigo = {"P": 0, "S": 1, "G": 2, "O": 3, "C": 4}
143         return np.array([[codigo[c] for c in fila] for fila in self.mapa])
144
145     def _render_matplotlib(self, ax=None):
146         cmap = ListedColormap(["#f2f2f2", "#4c9be8", "#3fa34d", "#3b3b3b", "#e07b39"])
147         creado = ax is None
148         if creado:
149             _, ax = plt.subplots(figsize=(5, 5))
150             ax.imshow(self._grid_codificado(), cmap=cmap, vmin=0, vmax=4)
151             ax.scatter(self.pos[1], self.pos[0], s=380, c="red", zorder=3, label="agente")
152             ax.set_xticks(np.arange(-0.5, self.n_cols, 1), minor=True)
153             ax.set_yticks(np.arange(-0.5, self.n_filas, 1), minor=True)
154             ax.grid(which="minor", color="white", linewidth=1.5)
155             ax.set_xticks(range(self.n_cols))
156             ax.set_yticks(range(self.n_filas))
157             ax.set_title(f"WarehouseEnv - paso {self.pasos} - pos {self.pos}")
158             ax.legend(loc="upper right")
159             if creado:
160                 plt.show()
161         return ax

```

2. Verificación del entorno

```

1 env = WarehouseEnv()
2
3 print("observation_space:", env.observation_space)
4 print("action_space      :", env.action_space)
5 print("inicio S:", env.inicio, "-> estado", env.a_estado(env.inicio))
6 print("meta   G:", env.meta, "-> estado", env.a_estado(env.meta))
7
8 estado, info = env.reset(seed=0)
9 print("\nreset() ->", estado, info)
10 print(env.render())

```

```

observation_space: Discrete(64)
action_space      : Discrete(4)
inicio S: (0, 0) -> estado 0
meta   G: (0, 7) -> estado 7

reset() -> 0 {'pos': (0, 0), 'congestion': False, 'pasos': 0}
paso 0/100   pos=(0, 0)
[A] ~ ~ ~ ~ ~ ~ G
. . . . .
. . . . .
. # # . # # # .
. . . . .
. # # # # # # .
. . . . .
. . . . .
. . . . .
A=agente S=recogida G=entrega #=obstaculo ~=congestion .=pasillo

```

1 # Un episodio corto con acciones fijas para comprobar step(), las recompensas y la terminación

```

1 # on episodio corto con acciones fijas para completar step(7), los recompensas y la termination.
2 env_demo = WarehouseEnv(render_mode="human")
3 env_demo.reset(seed=0)
4
5 for accion in [1, 3, 3, 0]: # abajo, derecha, derecha, arriba (arriba entra a congestion)
6     estado, r, terminated, truncated, info = env_demo.step(accion)
7     print(f"accion={env_demo.NOMBRES_ACCIONES[accion]:<10} estado={estado:<3} r={r:>6} "
8           f"congestion={info['congestion']} terminated={terminated} truncated={truncated}")

```

```

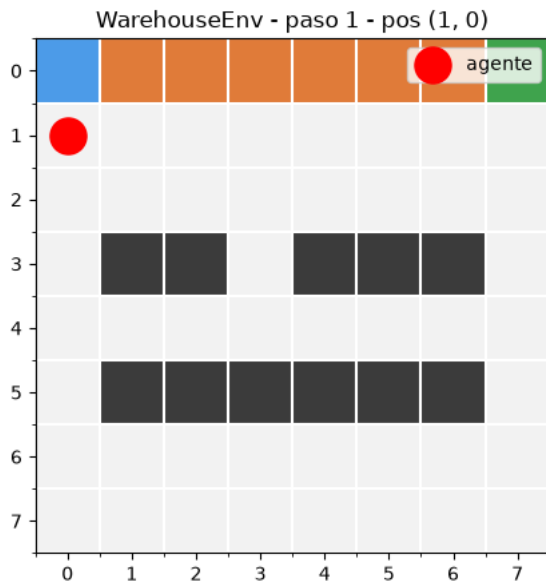
paso 0/100    pos=(0, 0)
[A] ~ ~ ~ ~ ~ G
. . . . .
. . . . .
. # # . # # # .
. . . . .
. # # # # # .
. . . . .
. . . . .
A=agente S=recogida G=entrega #=obstaculo ~=congestion .=pasillo
paso 1/100    pos=(1, 0)
S ~ ~ ~ ~ ~ G
[A] . . . . .
. . . . .
. # # . # # # .
. . . . .
. # # # # # .
. . . . .
. . . . .
A=agente S=recogida G=entrega #=obstaculo ~=congestion .=pasillo
accion=abajo estado=8 r= -1.0 congestion=False terminated=False truncated=False
paso 2/100    pos=(1, 1)
S ~ ~ ~ ~ ~ G
. [A] . . . . .
. . . . .
. # # . # # # .
. . . . .
. # # # # # .
. . . . .
. . . . .
A=agente S=recogida G=entrega #=obstaculo ~=congestion .=pasillo
accion=derecha estado=9 r= -1.0 congestion=False terminated=False truncated=False
paso 3/100    pos=(1, 2)
S ~ ~ ~ ~ ~ G
. . [A] . . . . .
. . . . .
. # # . # # # .
. . . . .
. # # # # # .
. . . . .
. . . . .
A=agente S=recogida G=entrega #=obstaculo ~=congestion .=pasillo
accion=derecha estado=10 r= -1.0 congestion=False terminated=False truncated=False
paso 4/100    pos=(0, 2)
S ~ [A] ~ ~ ~ ~ G
. . . . .
. . . . .
. # # . # # # .
. . . . .
. # # # # # .
. . . . .
. . . . .
A=agente S=recogida G=entrega #=obstaculo ~=congestion .=pasillo
accion=arriba estado=2 r= -20.0 congestion=True terminated=False truncated=False

```

```

1 # render en modo matplotlib
2 env_mpl = WarehouseEnv(render_mode="matplotlib")
3 env_mpl.reset(seed=0)
4 env_mpl.step(1) # baja a la ruta corta
5 env_mpl.render();

```



3. Algoritmos implementados desde cero

Ambos comparten la política de comportamiento ϵ -greedy y el mismo registro por episodio (recompensa total, pasos hasta la terminación y número de veces que el agente pasó por una zona de congestión). La única diferencia está en el *target* de la actualización:

- SARSA (on-policy): $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$ con a' la acción que realmente se ejecutará.
- Q-Learning (off-policy): $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$.

En ambos casos, cuando s' es terminal el target es simplemente r (el valor de un estado terminal es 0). El episodio truncado, en cambio, sí usa bootstrapping porque el estado no es terminal: solo se corta la interacción.

```

1 def politica_epsilon_greedy(Q, estado, epsilon, rng):
2     """Elige una accion epsilon-greedy. Los empates del argmax se rompen al azar."""
3     if rng.random() < epsilon:
4         return int(rng.integers(Q.shape[1]))
5     q = Q[estado]
6     return int(rng.choice(np.flatnonzero(q == q.max())))
7
8
9 def _registro_vacio():
10    return {"recompensa": [], "pasos": [], "congestiones": [], "exito": []}
11
12
13 def sarsa(env, n_episodios=8000, alpha=0.1, gamma=0.99, epsilon=0.2, seed=0):
14     """SARSA tabular on-policy. Devuelve (Q, historial_por_episodio)."""
15     rng = np.random.default_rng(seed)
16     Q = np.zeros((env.observation_space.n, env.action_space.n))
17     hist = _registro_vacio()
18
19     for _ in range(n_episodios):
20         estado, _ = env.reset()
21         accion = politica_epsilon_greedy(Q, estado, epsilon, rng)
22         total, pasos, congestiones = 0.0, 0, 0
23
24         while True:
25             sig_estado, r, terminated, truncated, info = env.step(accion)
26             total += r
27             pasos += 1
28             congestiones += int(info["congestion"])
29
30             if terminated:
31                 # El estado terminal no tiene valor futuro: el target es solo r.
32                 Q[estado, accion] += alpha * (r - Q[estado, accion])
33                 break
34
35             sig_accion = politica_epsilon_greedy(Q, sig_estado, epsilon, rng)
36             Q[estado, accion] += alpha * (r + gamma * Q[sig_estado, sig_accion] - Q[estado, accion])
37             estado, accion = sig_estado, sig_accion
38

```



```

39         if truncated:
40             break
41
42         hist["recompensa"].append(total)
43         hist["pasos"].append(pasos)
44         hist["congestionones"].append(congestionones)
45         hist["exito"].append(bool(terminated))
46
47     return Q, hist
48
49
50 def q_learning(env, n_episodios=8000, alpha=0.1, gamma=0.99, epsilon=0.2, seed=0):
51     """Q-Learning tabular off-policy. Devuelve (Q, historial_por_episodio)."""
52     rng = np.random.default_rng(seed)
53     Q = np.zeros((env.observation_space.n, env.action_space.n))
54     hist = _registro_vacio()
55
56     for _ in range(n_episodios):
57         estado, _ = env.reset()
58         total, pasos, congestionones = 0.0, 0, 0
59
60         while True:
61             accion = politica_epsilon_greedy(Q, estado, epsilon, rng)
62             sig_estado, r, terminated, truncated, info = env.step(accion)
63             total += r
64             pasos += 1
65             congestionones += int(info["congestion"])
66
67             objetivo = r if terminated else r + gamma * Q[sig_estado].max()
68             Q[estado, accion] += alpha * (objetivo - Q[estado, accion])
69             estado = sig_estado
70
71             if terminated or truncated:
72                 break
73
74             hist["recompensa"].append(total)
75             hist["pasos"].append(pasos)
76             hist["congestionones"].append(congestionones)
77             hist["exito"].append(bool(terminated))
78
79     return Q, hist

```

4. Entrenamiento

```

1 ALPHA, GAMMA, EPSILON = 0.1, 0.99, 0.2
2 N_EPISODIOS, SEED = 20000, 42
3
4 env = WarehouseEnv()
5
6 Q_sarsa, hist_sarsa = sarsa(env, N_EPISODIOS, ALPHA, GAMMA, EPSILON, seed=SEED)
7 Q_qlearning, hist_qlearning = q_learning(env, N_EPISODIOS, ALPHA, GAMMA, EPSILON, seed=SEED)
8
9 # Registro por episodio requerido para el analisis de Task 4.
10 train = pd.concat([
11     pd.DataFrame(hist_sarsa).assign(algoritmo="SARSA", episodio=lambda d: d.index),
12     pd.DataFrame(hist_qlearning).assign(algoritmo="Q-Learning", episodio=lambda d: d.index),
13 ], ignore_index=True)[["algoritmo", "episodio", "recompensa", "pasos", "congestionones", "exito"]]
14
15 print("Q-table SARSA      :", Q_sarsa.shape)
16 print("Q-table Q-Learning:", Q_qlearning.shape)
17 train.head()

```

Q-table SARSA : (64, 4)

Q-table Q-Learning: (64, 4)

	algoritmo	episodio	recompensa	pasos	congestionones	exito
0	SARSA	0	-379.0	62	22	True
1	SARSA	1	-138.0	100	2	False
2	SARSA	2	-44.0	69	4	True
3	SARSA	3	-100.0	100	0	False
4	SARSA	4	-100.0	100	0	False

```

1 resumen_train = train.groupby("algoritmo").agg(
2     recompensa_media=("recompensa", "mean"),
3     recompensa_ultimos_2000=("recompensa", lambda s: s.iloc[-2000:].mean()),
4     pasos_medios=("pasos", "mean"),
5     congestiones_totales=("congestiones", "sum"),
6     congestiones_por_episodio=("congestiones", "mean"),
7     tasa_exito=("exito", "mean"),
8 ).round(3)
9 resumen_train

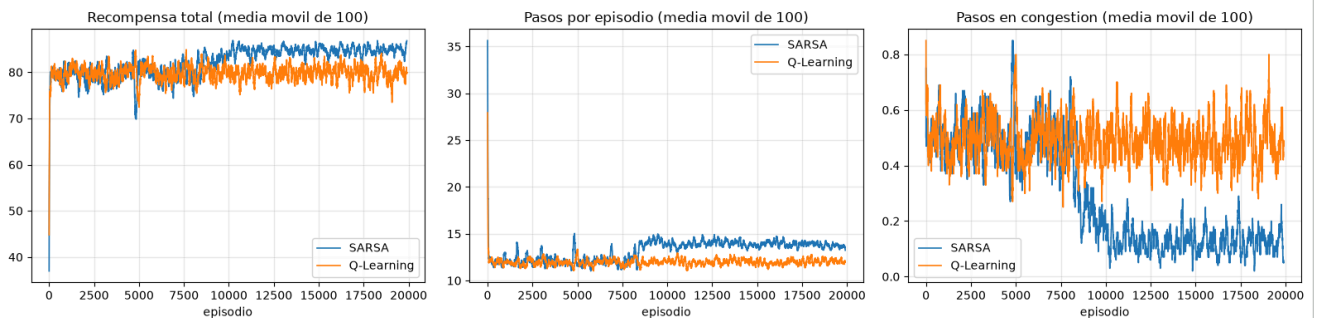
```

algoritmo	recompensa_media	recompensa_ultimos_2000	pasos_medios	congestiones_totales	congestiones_por_episodio	tasa_exito
Q-Learning	79.572	79.659	12.056	9801	0.490	0.999
SARSA	82.109	84.830	13.278	5828	0.291	0.999

```

1 def media_movil(x, k=100):
2     x = np.asarray(x, dtype=float)
3     return np.convolve(x, np.ones(k) / k, mode="valid")
4
5 fig, axes = plt.subplots(1, 3, figsize=(16, 4))
6 series = [("recompensa", "Recompensa total"), ("pasos", "Pasos por episodio"),
7     ("congestiones", "Pasos en congestión")]
8
9 for ax, (col, titulo) in zip(axes, series):
10     for nombre, hist in [("SARSA", hist_sarsa), ("Q-Learning", hist_qlearning)]:
11         ax.plot(media_movil(hist[col]), label=nombre, linewidth=1.2)
12     ax.set_title(f"{titulo} (media móvil de 100)")
13     ax.set_xlabel("episodio")
14     ax.grid(alpha=0.3)
15     ax.legend()
16
17 plt.tight_layout()
18 plt.show()

```



5. Evaluación con política greedy pura (100 episodios, $\epsilon = 0$)

```

1 def evaluar_greedy(env, Q, n_episodios=100, seed=123):
2     """Ejecuta la política greedy pura (epsilon = 0) y registra las mismas métricas."""
3     rng = np.random.default_rng(seed)
4     hist = _registro_vacio()
5
6     for _ in range(n_episodios):
7         estado, _ = env.reset()
8         total, pasos, congestiones = 0.0, 0, 0
9         while True:
10             accion = politica_epsilon_greedy(Q, estado, 0.0, rng)
11             estado, r, terminated, truncated, info = env.step(accion)
12             total += r
13             pasos += 1
14             congestiones += int(info["congestion"])
15             if terminated or truncated:

```

```

16         break
17     hist["recompensa"].append(total)
18     hist["pasos"].append(pasos)
19     hist["congestiones"].append(congestiones)
20     hist["exito"].append(bool(terminated))
21
22     return hist
23
24
25 eval_sarsa = evaluar_greedy(env, Q_sarsa, n_episodios=100)
26 eval_qlearning = evaluar_greedy(env, Q_qlearning, n_episodios=100)
27
28 evaluacion = pd.concat([
29     pd.DataFrame(eval_sarsa).assign(algoritmo="SARSA", episodio=lambda d: d.index),
30     pd.DataFrame(eval_qlearning).assign(algoritmo="Q-Learning", episodio=lambda d: d.index),
31 ], ignore_index=True)[["algoritmo", "episodio", "recompensa", "pasos", "congestiones", "exito"]]
32
33 resumen_eval = evaluacion.groupby("algoritmo").agg(
34     recompensa_medio=("recompensa", "mean"),
35     recompensa_std=("recompensa", "std"),
36     pasos_medios=("pasos", "mean"),
37     congestiones_por_episodio=("congestiones", "mean"),
38     tasa_exito=("exito", "mean"),
39 ).round(3)
40 resumen_eval

```

	recompensa_medio	recompensa_std	pasos_medios	congestiones_por_episodio	tasa_exito
algoritmo					
Q-Learning	92.0	0.0	9.0	0.0	1.0
SARSA	90.0	0.0	11.0	0.0	1.0

El entorno es determinista y el estado inicial es siempre el mismo, así que los 100 episodios de evaluación greedy son idénticos entre sí (desviación estándar 0). Se ejecutan igual porque la consigna lo pide y porque sirven para confirmar que ninguna de las dos políticas se queda atascada ni depende del azar del desempate: con $\varepsilon = 0$ y sin empates en el argmax, la trayectoria es completamente reproducible.

6. Políticas aprendidas

```

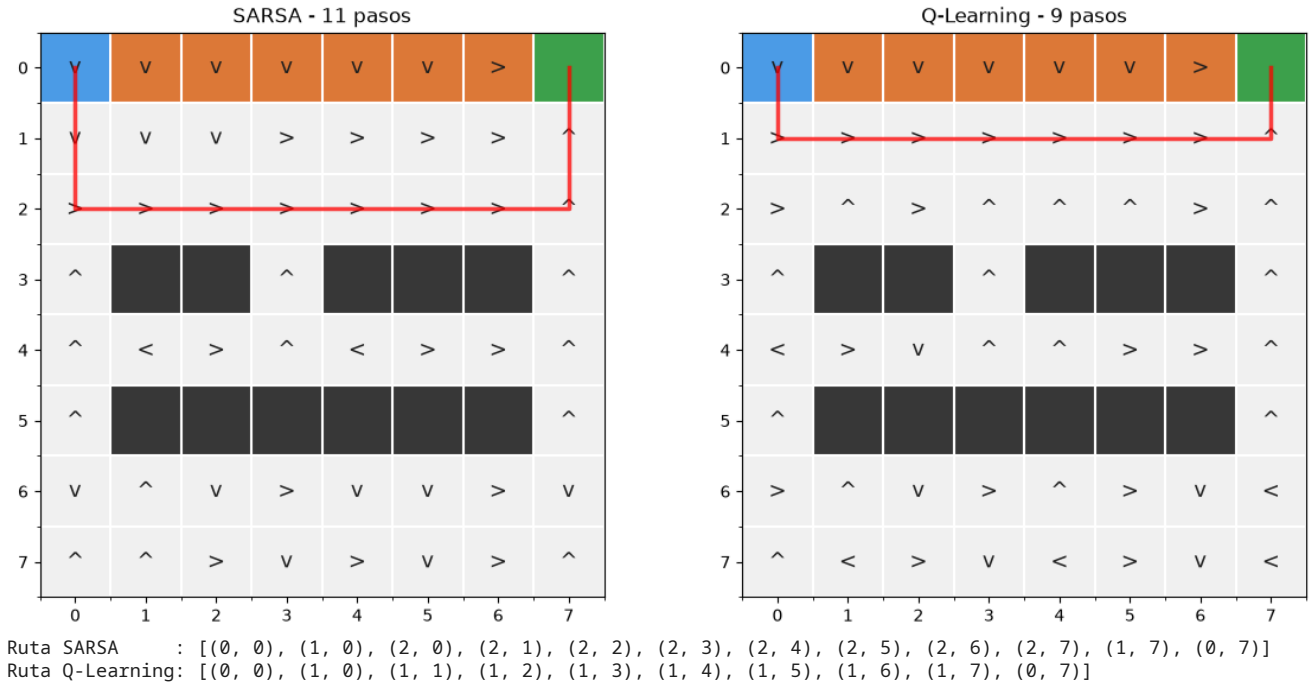
1 def ruta_greedy(env, Q):
2     """Trayectoria que sigue la politica greedy desde el inicio."""
3     estado, _ = env.reset()
4     rng = np.random.default_rng(0)
5     ruta = [env.pos]
6     while True:
7         accion = politica_epsilon_greedy(Q, estado, 0.0, rng)
8         estado, _, terminated, truncated, _ = env.step(accion)
9         ruta.append(env.pos)
10        if terminated or truncated:
11            return ruta
12
13
14 def dibujar_politica(env, Q, ruta, titulo, ax):
15     cmap = ListedColormap(["#f2f2f2", "#4c9be8", "#3fa34d", "#3b3b3b", "#e07b39"])
16     ax.imshow(env._grid_codificado(), cmap=cmap, vmin=0, vmax=4)
17
18     for estado in range(env.observation_space.n):
19         f, c = env.a_posicion(estado)
20         if env.mapa[f][c] in "OG":
21             continue
22         ax.text(c, f, env.FLECHAS[int(Q[estado].argmax())], ha="center", va="center",
23                fontsize=13, color="#222")
24
25     fs, cs = zip(*ruta)
26     ax.plot(cs, fs, color="red", linewidth=2.5, alpha=0.75, zorder=3)
27     ax.set_xticks(np.arange(-0.5, env.n_cols, 1), minor=True)
28     ax.set_yticks(np.arange(-0.5, env.n_filas, 1), minor=True)
29     ax.grid(which="minor", color="white", linewidth=1.5)
30     ax.set_xticks(range(env.n_cols)); ax.set_yticks(range(env.n_filas))
31     ax.set_title(titulo)
32
33

```

```

34 ruta_s = ruta_greedy(env, Q_sarsa)
35 ruta_q = ruta_greedy(env, Q_qlearning)
36
37 fig, axes = plt.subplots(1, 2, figsize=(12, 5.5))
38 dibujar_politica(env, Q_sarsa, ruta_s, f"SARSA - {len(ruta_s) - 1} pasos", axes[0])
39 dibujar_politica(env, Q_qlearning, ruta_q, f"Q-Learning - {len(ruta_q) - 1} pasos", axes[1])
40 plt.tight_layout()
41 plt.show()
42
43 print("Ruta SARSA      :", ruta_s)
44 print("Ruta Q-Learning:", ruta_q)

```



7. Registros exportados para Task 4

```

1 train.to_csv("metricas_entrenamiento.csv", index=False)
2 evaluacion.to_csv("metricas_evaluacion.csv", index=False)
3 np.save("q_sarsa.npy", Q_sarsa)
4 np.save("q_qlearning.npy", Q_qlearning)
5
6 print("metricas_entrenamiento.csv:", train.shape)
7 print("metricas_evaluacion.csv   :", evaluacion.shape)
8 print("q_sarsa.npy / q_qlearning.npy:", Q_sarsa.shape)

```

```

metricas_entrenamiento.csv: (40000, 6)
metricas_evaluacion.csv   : (200, 6)
q_sarsa.npy / q_qlearning.npy: (64, 4)

```

```

1 print(f"=== Entrenamiento ({N_EPISODIOS} episodios, epsilon = {EPSILON}) ===")
2 print(resumen_train.to_string())
3 print("\n=== Evaluacion greedy pura (100 episodios, epsilon = 0) ===")
4 print(resumen_eval.to_string())
5 print(f"\nCota teorica de la ruta optima con gamma={GAMMA}: "
6       f"{-(1 - GAMMA**8) / (1 - GAMMA) + 100 * GAMMA**8:.2f} (sin descuento: 92.00)")

```

```

=== Entrenamiento (20000 episodios, epsilon = 0.2) ===
recompensa_media  recompensa_ultimos_2000  pasos_medios  congestiones_totales  congestiones_por_episodio
algoritmo
Q-Learning        79.572                79.659         12.056           9801                0.490
SARSA              82.109                84.830         13.278           5828                0.291

=== Evaluacion greedy pura (100 episodios, epsilon = 0) ===
recompensa_media  recompensa_std  pasos_medios  congestiones_por_episodio  tasa_exito
algoritmo
Q-Learning        92.0             0.0           9.0             0.0                1.0
SARSA              90.0             0.0          11.0             0.0                1.0

```

Cota teorica de la ruta optima con gamma=0.99: 84.55 (sin descuento: 92.00)

8. Hallazgos de Task 3

Con $\alpha = 0.1$, $\gamma = 0.99$, $\varepsilon = 0.2$, 20000 episodios y semilla 42:

Métrica	SARSA	Q-Learning
Recompensa media de entrenamiento	82.11	79.57
Recompensa media, últimos 2000 episodios	84.83	79.66
Pasos medios por episodio (entrenamiento)	13.28	12.06
Pasos en congestión por episodio (entrenamiento)	0.291	0.490
Recompensa en evaluación greedy (100 episodios)	90.0	92.0
Pasos en evaluación greedy	11	9
Pasos en congestión en evaluación greedy	0	0

La predicción hecha en Task 2 se cumple en las dos direcciones:

- **En entrenamiento gana SARSA** (84.83 vs 79.66 en los últimos 2000 episodios). Al ser on-policy, incorpora en su target el riesgo de que la acción exploratoria lo tire a la banda de congestión, así que aprende a rodearla: pisa congestión 0.291 veces por episodio contra 0.490 de Q-Learning, un 41% menos.
- **En evaluación greedy gana Q-Learning** (92.0 vs 90.0). Sin exploración el entorno es determinista y no hay riesgo de resbalar, así que la ruta corta de 9 pasos que aprendió Q-Learning es efectivamente la óptima, y los 2 pasos extra del rodeo de SARSA se vuelven pérdida pura.
- Las políticas greedy son exactamente las dos rutas anticipadas: Q-Learning bordea la congestión por la fila 1 (9 pasos) y SARSA la rodea por la fila 2 (11 pasos).
- El valor de la ruta óptima en evaluación, 92.0, coincide exactamente con la cota teórica no descontada recalculada para este mapa ($8(-1) + 100$), lo que confirma que Q-Learning convergió al óptimo real.
- Ambas políticas alcanzan la meta en el 100% de los episodios de evaluación, y durante el entrenamiento la tasa de éxito es de 99.9% (los truncamientos ocurren casi solo en los primeros episodios, cuando la política es prácticamente aleatoria).

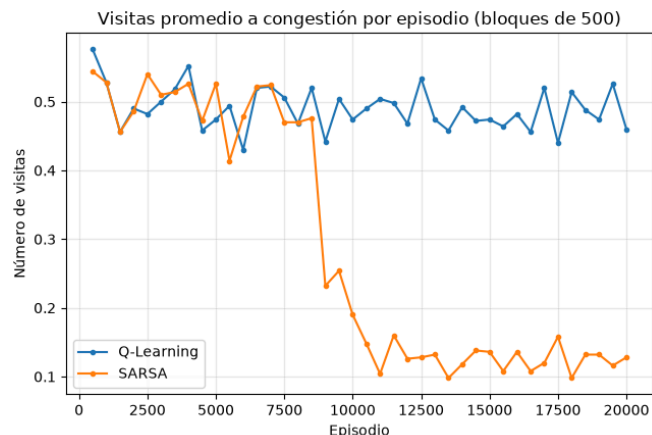
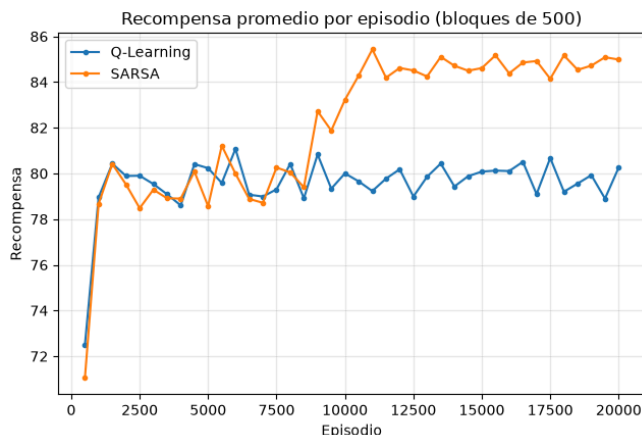
Task 4: análisis experimental y dictamen técnico

1-2. Verificación de predicciones, convergencia y comportamiento diferencial

```

1 # Promedios por bloques no solapados para reducir ruido sin ocultar la escala temporal.
2 TAM_BLOQUE = 500
3 bloques = (train.assign(bloque=lambda d: d['episodio'] // TAM_BLOQUE)
4             .groupby(['algoritmo', 'bloque'], as_index=False)
5             .agg(episodio=('episodio', 'max'), recompensa=('recompensa', 'mean'),
6                 congestiones=('congestiones', 'mean')))
7 fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
8 for nombre, datos in bloques.groupby('algoritmo'):
9     axes[0].plot(datos['episodio'], datos['recompensa'], marker='o', ms=3, label=nombre)
10    axes[1].plot(datos['episodio'], datos['congestiones'], marker='o', ms=3, label=nombre)
11 for ax, titulo, ylabel in zip(axes, ['Recompensa promedio por episodio', 'Visitas promedio a congestión por epis
12    ax.set(title=f'{titulo} (bloques de {TAM_BLOQUE})', xlabel='Episodio', ylabel=ylabel)
13    ax.grid(alpha=.3); ax.legend()
14 plt.tight_layout(); plt.show()
15 # Separación = 3 bloques seguidos con ventaja SARSA >=2 y >=0.10 visitas menos.
16 ancho = bloques.pivot(index='bloque', columns='algoritmo', values=['recompensa', 'congestiones'])
17 criterio = ((ancho['recompensa']['SARSA'] - ancho['recompensa']['Q-Learning'] >= 2) & (ancho['congestiones']['Q-
18 sostenido = criterio.rolling(3).sum().eq(3)
19 primer_bloque = int(sostenido.idxmax() - 2) if sostenido.any() else None
20 episodio_separacion = primer_bloque * TAM_BLOQUE if primer_bloque is not None else None
21 print('Inicio de separación sostenida:', episodio_separacion)
22 display(bloques[bloques['bloque'].isin(range(max(0, (primer_bloque or 0)-1), (primer_bloque or 0)+4))].round(3))
23

```



Inicio de separación sostenida: 9000

	algoritmo	bloque	episodio	recompensa	congestiones
17	Q-Learning	17	8999	80.832	0.442
18	Q-Learning	18	9499	79.334	0.504
19	Q-Learning	19	9999	80.002	0.474
20	Q-Learning	20	10499	79.654	0.490
21	Q-Learning	21	10999	79.228	0.504
57	SARSA	17	8999	82.724	0.232
58	SARSA	18	9499	81.876	0.254
59	SARSA	19	9999	83.224	0.190
60	SARSA	20	10499	84.284	0.148
61	SARSA	21	10999	85.428	0.104

Contraste de las predicciones de Task 2.

- Mayor recompensa durante entrenamiento para SARSA: confirmada.** En los últimos 2,000 episodios, SARSA obtuvo 84.83 frente a 79.66 de Q-Learning. Además, entró 0.291 veces por episodio en congestión frente a 0.490 (40.6% menos). Al ser *on-policy*, su target incorpora la siguiente acción realmente seleccionada por la política ε -greedy y el costo de una exploración accidental. Q-Learning actualiza hacia $\max_a Q(s', a)$ y aprende la ruta corta suponiendo conducta greedy futura, aunque durante el entrenamiento siga explorando.
- Mayor recompensa greedy para Q-Learning: confirmada.** Q-Learning consiguió 92 puntos y 9 pasos; SARSA, 90 puntos y 11 pasos. Ambas políticas tuvieron 100% de éxito y cero congestiones en 100 evaluaciones. Con $\varepsilon = 0$ desaparece el riesgo exploratorio y los dos pasos adicionales son costo puro. Q-Learning igualó la cota no descontada $8(-1) + 100 = 92$ (84.55 con descuento).
- Efecto de ε e identidad con $\varepsilon = 0$: parcialmente confirmada.** El caso $\varepsilon = 0.2$ confirma la diferencia prevista. La identidad en $\varepsilon = 0$ es asintótica bajo visitas suficientes; entrenar greedy desde cero no las garantiza. Cuando la conducta es greedy y se cumplen las condiciones de convergencia, SARSA usa la acción que maximiza Q y su target coincide con el de Q-Learning, por lo que ambos convergen a Q^* . La predicción matemática se sostiene, pero el protocolo principal no la prueba directamente.

La figura define visibilidad mediante un criterio reproducible: tres bloques consecutivos en los que SARSA aventaja por al menos 2 puntos y registra al menos 0.10 visitas menos. La primera separación sostenida comienza en el **episodio 9,000** y se mantiene en los bloques siguientes. El orden coincide con la teoría: tras descubrir rutas útiles, SARSA internaliza el riesgo exploratorio y se desplaza al pasillo seguro; Q-Learning conserva la ruta corta.

3. Análisis de sensibilidad de α y ε

Se prueban tres combinaciones con cinco semillas, 20,000 episodios por corrida y $\gamma = 0.99$. La tabla resume la media y desviación entre semillas de 100 evaluaciones greedy por tabla aprendida y cuántas semillas produjeron la ruta segura de 11 pasos.

```
1 COMBINACIONES = [(0.05, 0.10), (0.10, 0.20), (0.20, 0.30)]
2 SEMILLAS = [11, 22, 33, 44, 55]
3 resultados_sensibilidad = []
4 for alpha, epsilon in COMBINACIONES:
5     for semilla in SEMILLAS:
6         for nombre, entrenador in [('SARSA', sarsa), ('Q-Learning', q_learning)]:
7             Q, hist = entrenador(WarehouseEnv(), N_EPISODIOS, alpha, GAMMA, epsilon, seed=semilla)
8             ev = evaluar_greedy(WarehouseEnv(), Q, 100, seed=1000 + semilla)
9             resultados_sensibilidad.append({'alpha':alpha, 'epsilon':epsilon, 'algoritmo':nombre, 'semilla':semi
10 sensibilidad = pd.DataFrame(resultados_sensibilidad)
11 tabla_sensibilidad = (sensibilidad.groupby(['alpha','epsilon','algoritmo']).agg(recompensa_greedy_media=('recomp
12 tabla_sensibilidad
13
```

			recompensa_greedy_media	recompensa_greedy_std	recompensa_train_ultimos_2000	pasos_greedy
alpha	epsilon	algoritmo				
0.05	0.1	Q-Learning	92.0	0.000	86.768	9.0
		SARSA	92.0	0.000	86.883	9.0
0.10	0.2	Q-Learning	92.0	0.000	79.549	9.0
		SARSA	90.0	0.000	84.832	11.0
0.20	0.3	Q-Learning	92.0	0.000	70.840	9.0
		SARSA	52.0	84.971	79.798	28.8

Recomendamos $(\alpha, \epsilon) = (0.10, 0.20)$ **para SARSA**: produjo la ruta segura de 11 pasos en las cinco semillas, alcanzó 90.0 puntos greedy sin variación y obtuvo 84.832 puntos de entrenamiento tardío. Con $(0.05, 0.10)$ ambos algoritmos aprendieron la misma ruta corta de 92 puntos: la exploración no fue suficiente para que SARSA valorara el riesgo diferencial. Con $(0.20, 0.30)$ SARSA eligió rutas de al menos 11 pasos en las cinco semillas, pero fue inestable (52.0 puntos greedy, desviación 84.971 y 28.8 pasos medios), mientras el costo exploratorio redujo su recompensa tardía a 79.798. Q-Learning obtuvo 92 puntos y 9 pasos en todas las combinaciones, aunque su recompensa de entrenamiento cayó de 86.768 a 70.840 al aumentar ϵ . En producción no se mantendría exploración aleatoria: se entrenaría y validaría fuera de línea y se desplegaría una política greedy con capa de seguridad.

4. Investigación bibliográfica

Ajabshir, Güzel y Bostanci (2022) estudian navegación descentralizada de múltiples robots en espacios continuos parcialmente obstruidos: cada agente debe alcanzar su objetivo evitando obstáculos y colisiones sin controlador central. Aplican Q-Learning *model-free*, pero atacan una limitación presente en este laboratorio: el crecimiento combinatorio de la tabla al agregar variables. Separan características no superpuestas y comprimen la representación (CCSS); después añaden priorización de objetos y reducción de datos sensoriales en un controlador generalizado (GCC). Discretizan observaciones continuas, moldean la recompensa para favorecer acercarse a la meta y reducen gradualmente la exploración.

Reportan cerca de **70% menos memoria**, navegación con al menos 18 robots y soluciones cercanas al óptimo. Para tres robots, CCSS y el controlador no comprimido muestran curvas similares y una solución razonable después de unos 400,000 episodios: la ventaja central es memoria y escalabilidad, no una nueva regla TD. El método mitiga la explosión del espacio y admite observaciones continuas mediante discretización y selección de características. No elimina, sin embargo, la alta demanda de interacciones, la dependencia de discretización, recompensa e hiperparámetros, la falta de seguridad durante exploración ni la limitada generalización. El traslado a Deep Q-Learning queda como trabajo futuro y tampoco sustituye validación física con ruido sensorial y dinámica real.

5. Dictamen técnico para gerencia

Recomendamos **SARSA para aprender una política destinada al robot real** si existe exploración en línea o si entrar en congestión implica riesgo humano, daño o interrupción. La evidencia: en los últimos 2,000 episodios obtuvo 84.83 puntos frente a 79.66 de Q-Learning y redujo las visitas a congestión de 0.490 a 0.291, a cambio de una ruta greedy de 11 pasos y 90 puntos. Q-Learning se recomienda solo si todo el aprendizaje ocurre en un simulador representativo, la política se congela antes del despliegue y prima el rendimiento: alcanzó la cota óptima de 92 puntos en 9 pasos, sin congestión en evaluación greedy. Antes de producción se requieren tráfico dinámico y ruido en un gemelo digital, múltiples inicios y entregas, más semillas, transferencia simulación-real, sensores redundantes, parada de emergencia, restricciones explícitas y un piloto supervisado.

Referencias (APA 7.ª edición)

Ajabshir, V. B., Güzel, M. S., & Bostanci, E. (2022). A low-cost Q-learning-based approach to handle continuous space problems for multi-robot navigation in partially known environments. *Robotics and Computer-Aided Design*, 155, 102553. <https://doi.org/10.1016/j.rcad.2022.102553>